

BUILDING THE CENTER OF EXCELLENCE FOR POWER PLATFORM

WHY POWER PLATFORM PARTNERS MUST EVOLVE

**RETHINKING VALUE, TALENT AND THE CONSULTING ROLE IN THE
AGENTIC CENTER OF EXCELLENCE**

WHITE PAPER - SEPTEMBER 2026

NICOLÁS FERNÁNDEZ

Why Power Platform Partners Must Evolve

Rethinking value, talent and the consulting role in the agentic Center of Excellence

© September 2026 Nicolás Andrés Fernández. All rights reserved.

First edition, September 2026.

ABOUT THE AUTHOR

Nicolás Andrés Fernández specializes in Microsoft Power Platform, Copilot Studio, and Dynamics 365. His contributions to the Microsoft ecosystem have earned him seven consecutive Microsoft Most Valuable Professional (MVP) awards. With more than 17 years of experience, he has designed, developed, and implemented low-code and CRM solutions for organizations across multiple industries and countries. He is the author of the Building the Center of Excellence for Power Platform book series and has extensive experience creating frameworks for adoption, governance, innovation, and sustainable platform management. Nicolás regularly shares his expertise as a speaker at Microsoft and community events, while also supporting initiatives that help organizations and professionals turn technology into sustainable business value.

COLLABORATORS

Technical reviewer: Franco Martino Gagliardi & Wilmer Alcivar

This white paper is independently published by Nicolás Andrés Fernández. It is not official guidance from Microsoft or any other vendor and does not replace official product documentation, licensing terms, security guidance, professional advice, or the organization's own legal, risk, privacy, compliance, and Responsible AI decisions. Microsoft, Power Platform, Microsoft Copilot Studio, and other product names referenced in this document are trademarks of their respective owners. References to those products are used only for identification and educational discussion.

A partner should be measured by how much more capable the Center of Excellence becomes because it is there.

Contents

Prologue	4
Part I: Why the Partner Model Must Change.....	7
1. The Partner Has a Relevance Problem Before It Has an AI Problem.....	8
2. The Traditional Partner Model Was Built Around Scarcity	10
3. Power Platform Has Always Been Moving Toward Abundance	12
Part II: Rethinking Partner Value Across the Agentic Center of Excellence	14
4. The CoE as the Map of Future Partner Value	15
5. Strategy & Advisory - Moving Upstream to the Design of Work	17
6. Center of Governance - From Managing Controls to Governance Intelligence.....	19
7. The Innovation Hub - From Abundant Ideas to Industrialized Production.....	21
8. Supervised Design - The Discipline Between Consulting and Industrialized Delivery.....	25
9. Center of Enablement - Sustainable Adoption Is Not a Training Schedule.....	27
10. Sustain - From Brute Force to Agentified Operations	29
Part III: Rethinking Talent	31
11. Human Value Moves Toward Ambiguity, Judgment, and Accountability	32
Part IV: Rethinking Consulting	38
12. The Return of Consulting and the Rise of Executable Consulting IP	39
13. Commercial Models Need to Reward Leverage Rather Than Effort.....	42
Part V: The Partner Transformation Agenda	44
14. Partners Need to Become Customer Zero of Their Own Operating Model.....	45
15. Three Stages of Partner Evolution.....	47
16. Metrics Will Reveal Whether the Partner Has Really Changed	49
17. The Transition Should Begin Before the Old Model Stops Working.....	50
Conclusion - The Partner as a Force Multiplier for the Agentic Center of Excellence	52
Continue the Series.....	54

Prologue

For most of the history of enterprise technology consulting, Microsoft partners have operated in an environment defined by scarcity: organizations required specialized technical capabilities that were difficult to build and retain internally, while transformation programs routinely generated more implementation demand than internal teams could absorb. Partners filled that gap by supplying expertise, delivery capacity, and operational support, and the model was not merely commercially successful but a rational response to the economics of enterprise software.

Within the Power Platform ecosystem that model survived the arrival of low-code, because although the platform reduced the amount of conventional code required to build applications and automations, it simultaneously expanded the addressable market: organizations discovered more processes worth digitizing, more employees became capable of participating in solution creation, and partners found growing demand for implementation, governance, adoption, and managed services. Low-code changed the composition of the work without removing the need for large amounts of human delivery capacity.

Generative and agentic AI introduce a more consequential shift, because they do not simply lower the technical barrier to development but increasingly participate in the work partners themselves perform. Requirements can be analyzed and structured with AI assistance, solution structures can be generated from a natural-language description, code can be produced and reviewed by coding agents, tests and documentation can be generated, and operational incidents can be classified, diagnosed, and in some cases remediated without the same level of human intervention, while agents retrieve knowledge, invoke tools, and execute multi-step activities that previously required people to interpret information and coordinate actions.

This creates an economic problem for any firm whose value proposition remains tied to the amount of human effort required to implement and operate technology, because a partner can use AI to deliver the same services faster while leaving that proposition untouched. The gain shows up as margin at first, but productivity becomes an expectation rather than an advantage: customers learn that delivery is faster, competitors acquire similar tools, procurement adjusts its benchmarks, and the premium attached to executing faster erodes.

The question this white paper sets out to answer is therefore not how partners should use AI to improve delivery, which is by now a reasonably well-understood exercise, but a harder and less comfortable one: why a customer will need this partner at all once execution stops being scarce. That is a question about relevance rather than about technology, and it has to be answered before any of the others become interesting, because a firm can become excellent at Copilot Studio and still be losing the reason it was hired in the first place. If the customer can generate more of the solution, if the platform can propose more of the design, and if the operation can increasingly diagnose and repair itself, then the partner has to earn its place in the room again rather than inherit it from a shortage of skills.

The answer begins from the observation that the Center of Excellence is the most useful lens through which to look. A mature Power Platform and Copilot Studio CoE is already responsible for far more than governance, connecting strategic direction with innovation, enablement, delivery, and sustainable operations, and every one of those capabilities has to evolve as the organization enters the agentic era: the CoE must decide where agents should participate in business processes and where human accountability must remain, govern new forms of autonomy and tool use, cope with a pipeline of potential innovation that expands as experimentation gets cheaper, enable employees to collaborate with increasingly capable digital workers rather than merely to operate software, and redesign platform operations so that growth in the estate does not produce proportional growth in operational headcount.

The thesis that follows is that the partner must move in the same direction, and that the movement has a name: from capacity augmentation to capability augmentation. The traditional partner created value by adding people to whatever the customer was already doing, supplying developers when the backlog grew, service resources when support demand grew, trainers when adoption needed acceleration, and administrators when governance became more complex. The agentic partner creates value by helping the customer redesign the system that generates the demand, so that the answer to more work is not automatically more people. That single movement is what the rest of this paper develops, capability by capability, role by role, and commercial model by commercial model, and it is why the future partner should be judged by how much more capable the Center of Excellence becomes because the partner is there. That is also the answer to the relevance question, because a partner earns its place not by holding knowledge the customer lacks but by making the customer measurably better at what it is trying to do.

Relationship to the Center of Excellence series

This white paper is a companion to my Building the Center of Excellence for Power Platform series, in which I establish the broader organizational framework required to govern, enable, innovate, deliver, and sustain Power Platform and Copilot Studio at enterprise scale, treating the Center of Excellence as an operating model rather than a single team and extending that model into the agentic era, where autonomy, knowledge, actions, lifecycle, risk, and accountability must be considered alongside traditional low-code governance and adoption. The purpose here is deliberately different: this paper does not redefine the Center of Excellence or repeat the guidance for establishing its capabilities, but asks instead what the evolution of the CoE means for the Microsoft partner ecosystem.

The term Agentic Center of Excellence is used throughout as a conceptual description of a CoE operating in an environment in which applications, copilots, agents, and increasingly autonomous digital capabilities form part of the enterprise technology landscape; it is not a Microsoft product label, and it does not replace the established CoE framework. The CoE remains the CoE, and what changes is the environment in which it operates, and therefore the type of value a partner must bring to it.

WHAT THIS IS

A strategic white paper about how Power Platform partners should rethink value, talent, consulting, intellectual property, and commercial models as the Center of Excellence enters the agentic era. What this is not: A prediction that developers disappear, an official Microsoft partner framework, a licensing guide, or a step-by-step implementation manual for establishing a Center of Excellence.

Who should read it and how

The primary audience is Microsoft partner leadership: CEOs and managing directors, Power Platform and Business Applications practice leaders, Copilot Studio and AI practice leaders, consulting and delivery directors, CTOs and chief architects, managed-service leaders, and Microsoft alliance executives. It is also relevant to mature customer-side CoE leaders who want to understand how the role of their strategic partners should evolve.

The argument is organized around three lenses. Value asks where differentiation moves when execution becomes less scarce, Talent asks which human capabilities become more important when copilots, skills, and agents can absorb more repetitive implementation and operational work, and Consulting asks how the partner must reposition itself around a Center of Excellence that increasingly needs help redesigning work, managing innovation portfolios, governing autonomy, and continuously optimizing an intelligent platform estate.

Readers interested primarily in the economic thesis can focus on Chapters 1 through 3. Chapters 4 through 10 examine how partner value changes across the capabilities of the Center of Excellence, with Chapter 8 introducing Supervised Design as the discipline that connects them. Chapter 11 explores the implications for talent, and Chapters 12 and 13 those for consulting intellectual property and commercial models. Chapters 14 through 17 address the transformation agenda, the stages of evolution, the metrics, and the timing. The conclusion brings the argument together around a single proposition: the future partner should be evaluated by how much more capable the Center of Excellence becomes because the partner is there.

NOTE

The concepts of Capacity Augmentation, Capability Augmentation, Supervised Design, Governance Intelligence, Executable Consulting IP, Agent Experience Design, and the Capability Engineer role are analytical constructs used in this white paper. They are not presented as official Microsoft terminology, and the role names are not Microsoft job titles.

Part I: Why the Partner Model Must Change

1. The Partner Has a Relevance Problem Before It Has an AI Problem

Technology consulting has always evolved alongside the platforms it implements: when new products emerge partners create practices around them, when technical architectures change partners retrain their people, and when vendors release new capabilities consulting firms develop new propositions and recruit the specialists capable of implementing them. That rhythm of platform change and partner adaptation has been consistent enough that generative AI can easily be read as simply the latest turn of the same cycle.

Under that reading the response seems obvious. Power Platform partners should build Copilot Studio practices, teach developers to work with AI coding assistants, develop agent architects, produce demonstration use cases, and add agent implementation to existing service portfolios, while internally using AI to make presales, project delivery, testing, and support more efficient, and the partners that move faster than their competitors will gain a temporary productivity advantage and capture emerging demand. All of those actions are reasonable, and none of them addresses the deeper structural issue.

Agentic AI is unusual because it can participate in the professional work that consulting firms sell, which makes it not simply the object of the engagement but part of the consulting workforce. A new ERP product creates demand for people who understand how to implement that ERP product, a new cloud platform creates demand for people who understand cloud architecture and migration, and agentic technology certainly creates analogous demand for agent architects, AI engineers, and governance specialists; but it also changes how discovery is performed, how software is created, how testing is conducted, how operational work is executed, and how knowledge is reused. The same innovation therefore creates a new market while simultaneously reducing the effort required in parts of the existing one.

This is why the real challenge is not technological adoption but relevance. A partner can become exceptionally competent at Copilot Studio and still preserve an operating model built around selling large amounts of human implementation effort, can adopt every available productivity tool and still measure success primarily through billable utilization, and can automate significant parts of delivery while continuing to hire and organize its workforce as if every future engagement will require the same delivery pyramid as the previous decade. That partner will have adopted AI without adapting to the economics AI creates.

The more useful starting question is therefore not how our Power Platform practice should adopt AI, but why a customer will need our Power Platform practice in the future. Historically the answer was straightforward, because the partner possessed technical capabilities the customer lacked: it knew how to configure the platform, develop the solution, perform the integration, deploy the application, or support the estate, and when those skills are scarce the proposition is powerful.

As the skills become easier to access, or increasingly augmented by AI, the answer has to become richer. The customer may need a partner because it does not fully understand the process it is attempting to automate. It may need a partner because its internal teams are too close to the problem to challenge

established assumptions. It may need a partner because it does not know which of hundreds of possible agent opportunities are commercially meaningful. It may need a partner because it lacks a coherent architecture for a rapidly growing ecosystem of applications, automations, and agents. It may need a partner because new forms of autonomy introduce risks that existing governance models were never designed to control. It may need a partner because technology adoption is high while measurable business value is low. Those are different reasons to buy consulting, and they are considerably harder to commoditize.

The core argument is therefore not that the Power Platform partner ecosystem is becoming less important, but that the reasons the ecosystem is important are changing. A partner that continues to define itself primarily through its ability to execute implementation may remain busy for years while its strategic differentiation grows steadily more vulnerable, whereas a partner that moves toward understanding, judgment, design, architecture, and continuous optimization can build a more defensible position, because those capabilities remain context-dependent and resist reduction to generic automation.

This is also why the transformation cannot be delegated to an AI practice: it affects the identity of the firm, changing which services sales teams lead with, which roles are promoted, which metrics are rewarded, which assets are treated as intellectual property, and how the firm thinks about margin. A practice can be technically AI-enabled while the company remains economically attached to the old model, which is why the transition belongs at executive level, as a question of what the partner intends to be when implementation is no longer the only scarce resource in the room.

SPOTLIGHT

Power Platform partners do not simply need an AI strategy. They need a relevance strategy.

2. The Traditional Partner Model Was Built Around Scarcity

To understand why partners must evolve it is important not to caricature the traditional services model, which did not emerge because consulting firms were inefficient or because customers irrationally preferred large teams, but because enterprise technology was genuinely difficult to implement and operate.

A complex business application historically required several distinct forms of expertise. Functional specialists needed to understand how the platform represented business processes, developers were required for extensions and integrations, testers had to validate combinations of configuration and custom functionality, release specialists managed deployment between environments, administrators operated the platform after go-live, and support teams diagnosed failures that demanded knowledge of both the business solution and its underlying technology. As programs grew, internal organizations rarely possessed enough people across all those disciplines to absorb peaks in demand, so partners provided elasticity: they could assemble teams more quickly, apply expertise accumulated across multiple customers, and supply specialists that would be inefficient for a single organization to retain permanently.

The commercial model reflected that reality, translating complexity into effort, effort into roles, roles into capacity, and capacity into price. Even when an engagement was fixed price its economics were usually based internally on an estimate of human effort, so a project might be sold as an outcome while the margin calculation beneath the proposal still depended heavily on the expected number of consultant days required to produce it.

The same logic applied to managed services, where operational demand grew with the application estate: additional environments required administration, additional users generated support requests, additional integrations created more potential points of failure, and additional solutions created more release and lifecycle activity, all of which partners monetized by supplying operational capacity. In that context headcount was not simply an organizational cost but productive capacity.

This helps explain why AI productivity creates tension for professional-services organizations. In a product company, reducing the labor required to manufacture each additional unit is clearly beneficial because the customer is buying the product rather than the hours required to produce it; in a consulting model that prices primarily through effort, reducing effort can reduce the unit being sold.

Initially the firm may capture the productivity gain as margin, since a consultant who once required five days to perform an activity may complete it in two while the customer still pays the existing project price. That advantage does not remain invisible indefinitely: customers learn that delivery is faster, competitors adopt similar capabilities, procurement functions adjust their benchmarks, and vendors themselves encourage customers to expect productivity improvements, until yesterday's AI advantage becomes tomorrow's expected baseline.

This pressure is widely discussed within the Microsoft partner ecosystem as market compression, and Microsoft's own guidance to partners has increasingly emphasized transforming service offerings, intellectual property, talent models, and commercial approaches. It is not a prediction that demand disappears, but a statement that the economic value attached to a particular unit of labor can decline when the same outcome requires less of that labor.

Power Platform partners have already experienced a version of the effect through low-code, where routine development that would once have required significant custom code became faster and yet the market expanded, because organizations discovered more demand than before. That history should make partners optimistic about the size of the opportunity and cautious about assuming that value will stay in the same place: the platform is moving toward abundance, and partner strategy has to move toward the next scarcity.

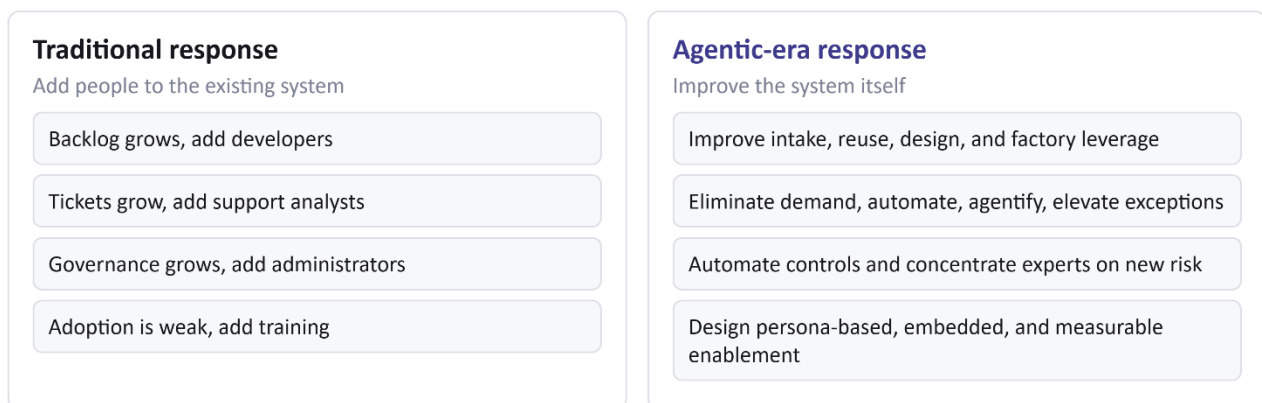


Figure 1. From capacity augmentation to capability augmentation.

The shift is not from people to no people. It is from adding effort to the existing system toward improving the system itself.

3. Power Platform Has Always Been Moving Toward Abundance

Power Platform partners are particularly exposed to this discussion because the platform itself has always been built around the democratization of technology creation. The original low-code proposition challenged the assumption that every business application required a professional development team: makers could construct applications using declarative tools, combine data sources through connectors, and automate processes without writing the same quantity of conventional code, and while professional developers remained essential for more complex requirements, routine solution creation became accessible to a far larger population.

That did not destroy the partner market; on the contrary, it contributed to its growth, because when the cost of producing solutions fell organizations discovered far more problems worth solving. Demand expanded beyond the finite list of enterprise applications that central IT could justify developing, departments began automating local processes, fusion teams emerged, citizen development created new governance requirements, and organizations needed help establishing environments, policies, ALM, communities, adoption programs, and CoEs.

The historical pattern matters because it suggests that reducing implementation scarcity does not necessarily reduce overall market opportunity, although it does change where the opportunity sits. Low-code moved some value away from writing code and toward solution design, governance, enablement, and platform management, and agentic technology continues the same movement at a higher level of abstraction.

The maker of the low-code era still had to understand how to create the application, knowing how to design screens, structure data, configure formulas, and create flows, whereas increasingly the user can describe the business problem and ask the platform to participate in determining what should be created. The platform is steadily acquiring capabilities that use generative AI to assist in the planning and creation of solutions, so that a description of a business problem, together with supporting material, increasingly contributes to the design of an end-to-end solution spanning data, applications, automation, and agents, while agent behavior is increasingly encapsulated into reusable units rather than reconstructed manually for every agent. The abstraction boundary is moving from "how do I code this?" to "how do I configure this?" and now increasingly toward "what outcome am I trying to achieve?"

This is a profound change because intent sits much closer to the business problem than implementation does, and if the platform itself can translate a growing portion of intent into artifacts, then the partner cannot build its future differentiation exclusively on a superior ability to produce those artifacts by hand. The partner has to move closer to the source of intent, which means understanding whether the intent is valid, understanding the business context, recognizing whether the request represents a symptom rather than a root cause, determining which implementation pattern is appropriate, establishing the constraints within which generation should occur, and evaluating whether the output is safe, sustainable, and valuable.

The lower layers do not disappear but become increasingly abstracted, and the partner's value has to climb the abstraction stack with them.

That movement also helps explain why the Center of Excellence becomes such a useful reference point, since a CoE exists precisely because democratized technology creates organizational questions that technology alone does not answer: when more people can build, somebody must define how the organization governs, prioritizes, enables, standardizes, reuses, and sustains that capability, and agentic AI intensifies every one of those questions. The partner that understands the CoE as an operating model can therefore move with the abstraction, stop thinking of the CoE as a source of implementation work, and begin treating it as the system whose capabilities the partner exists to strengthen.

Part II: Rethinking Partner Value Across the Agentic Center of Excellence

4. The CoE as the Map of Future Partner Value

The phrase "partners must become more strategic" is common in periods of technology change, and it is directionally useful while remaining operationally vague. Strategic in what sense? Which services change? Where should new skills be applied? How does the partner's role interact with the internal customer organization?

The Center of Excellence provides a concrete framework for answering those questions, because it already represents the organizational system through which Power Platform is governed, adopted, developed, and sustained. A mature CoE is not a single governance team but a set of capabilities that connects strategic direction with the practical realities of innovation, enablement, delivery, and operations.

An agentic CoE does not replace those responsibilities; it increases their scope. If solution creation becomes easier, innovation portfolio management becomes more important; if agents can take actions, governance becomes more sophisticated; if technology changes more quickly, enablement has to become continuous; if an organization can create substantially more digital capabilities, operations must become more scalable; and if implementation is increasingly generated, design and architecture become more important, because errors propagate faster.

The central partner question can therefore be reframed capability by capability. How should the partner create value within Strategy & Advisory? How should it contribute to Governance when agents introduce new forms of autonomy? What does the Center of Innovation require when ideas become cheap to prototype? Where is value in the Innovation Hub if implementation becomes increasingly generated? What does sustainable Enablement mean when employees need to collaborate with agents rather than merely learn software? How does Sustain scale when the estate includes increasingly autonomous digital capabilities?

The first of those names needs a word of explanation, because the CoE's strategy layer is, in my framework, the steering committee and the strategic vision area that keep platform investment aligned to business objectives. Since the argument here looks at it from the outside, it is named Strategy & Advisory: the capability inside the CoE, and the advisory service a partner brings to it.

That perspective produces a much richer analysis than asking which technical roles AI will automate, and it also keeps the Center of Innovation and the Innovation Hub in their proper relationship rather than collapsing them into a single thing. The Center of Innovation establishes the innovation themes, the criteria, and the evidence a proposal must produce before it can consume investment, while the Innovation Hub is the executing arm that runs the funnel against those criteria and carries what survives through into production. The agentic era raises the stakes at both ends, which is why Chapter 7 treats them together.

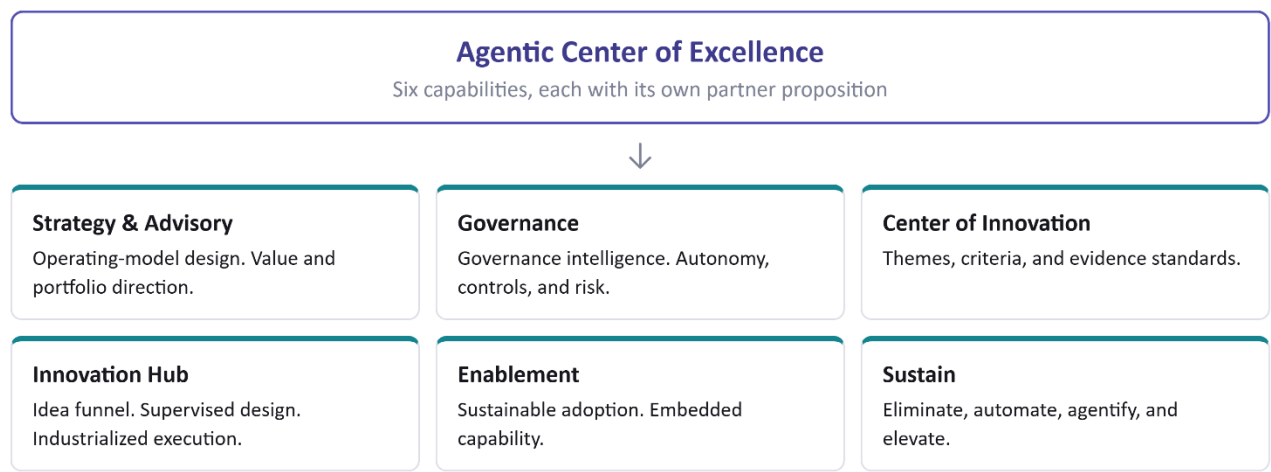


Figure 2. The CoE as the map of future partner value.

The partner's value proposition should be assessed against each CoE capability, not only against project implementation.

5. Strategy & Advisory - Moving Upstream to the Design of Work

The strategic partner needs to become involved before a technology decision has hardened into a requirement. Traditional technology engagements often begin with a sentence such as "We need an application for this process", or increasingly "We want an agent for this use case", and by the time the partner receives that request the most consequential assumption may already have been made: that the existing process should continue, and that a particular category of technology is the correct answer. A transformation partner should challenge both of them.

This becomes more important as agentic technology expands the solution design space, because a business activity can now be handled in several ways. It may remain a human responsibility, it may be supported by a conventional user interface, it may be automated deterministically through Power Automate, it may be assisted by a conversational copilot, it may be delegated partly to an agent that retrieves information and invokes tools, or it may be redesigned entirely because the technology has removed the constraint that caused the process to exist in its current form. Selecting among those patterns requires more than platform knowledge: it requires understanding the economics of the process, the decisions embedded within it, the degree of acceptable autonomy, the nature of exceptions, the risk of incorrect action, and the experience expected by employees or customers.

This is where the partner needs to recover a stronger consulting posture, because the first conversation should not be about what to build but about what work the organization is trying to improve, and that shift brings process analysis, operating-model design, and business architecture into the core of the partner proposition.

Consider a sales process, which should not be agentified simply because an agent can generate account research or update a CRM record. The strategic question is how sellers should spend their time, which information they require at each stage, what administrative activities should disappear, which decisions require relationship judgment, and where an agent can reduce friction without weakening accountability, and the technology design should follow from that answer.

The same principle applies to service, where an organization may ask for an agent to route customer requests because routing is visible and measurable, while the real opportunity may be to eliminate routing altogether by allowing a single agentic layer to gather context across systems and resolve a larger proportion of requests without organizational hand-offs. A partner that simply automates the existing routing tree makes the wrong process faster rather than making the operating model better.

The Strategy & Advisory capability should therefore include process transformation, business-capability analysis, agent opportunity assessment, portfolio strategy, value modeling, operating-model design, and technology positioning, so that the partner can explain not only what Microsoft technology can do but what the organization should do with it.

That creates a different relationship with senior stakeholders, since a partner leading with implementation estimates tends to engage after the strategic decisions have been made, whereas a partner that can credibly facilitate process redesign and value assessment can participate before budgets and solution choices are locked, and that earlier position is where much of the future differentiation lies.

There is a commercial consequence as well. Upstream consulting is harder to commoditize than downstream implementation because the work depends so heavily on organizational context: two companies using the same Power Platform products may require different operating models, different autonomy boundaries, and different change strategies. Product knowledge is portable while context is not, and so the partner moves from being an expert in the platform to being an expert in the relationship between the platform and the organization.

STRATEGIC SHIFT

The partner of the previous era answered, "How do we implement Power Platform?" The partner of the agentic era must answer, "How should Power Platform, Copilot Studio, and agents participate in the operating model we are trying to create?"

6. Center of Governance - From Managing Controls to Governance Intelligence

Governance has always been one of the strongest areas of partner opportunity around Power Platform, because democratized technology creates a structural tension between innovation and control: the platform enables large populations to create solutions, and the organization must ensure that those solutions comply with security, data, lifecycle, and architectural expectations.

The introduction of agents increases that tension because the governed object itself changes. A traditional application executes explicitly implemented logic, and the range of actions it can perform is generally bounded by what developers and makers configured, whereas an agent's behavior may depend on instructions, knowledge, context, tools, and runtime reasoning, so that it may choose among available tools based on an interpreted objective, work across multiple steps, interact with other agents, and perform activities on behalf of users. The resulting governance surface is broader than conventional application governance: data policies, environment strategy, sharing, identity, ALM, and connector controls remain relevant, but they are joined by questions about autonomy, tool combinations, behavioral evaluation, agent identity, knowledge boundaries, human approval, and accountability.

For the partner, the most important opportunity is not implementing the controls that Microsoft provides, because those controls will continue to evolve, product interfaces will become easier to administer, platform recommendations will improve, and some governance execution will itself become automated. The scarce skill will be recognizing when the organization's governance model must change.

Imagine that a new capability allows an agent to reach a class of enterprise systems that were previously unavailable: the technical question is how the connection is configured, while the governance question is whether the existing policy model adequately considers what the agent could now do with that access. Or imagine that an agent can initiate a commercial transaction, in which case the organization must decide what transaction value requires human approval, what evidence should be retained, how exceptions should be monitored, and who is accountable when the agent acts within its configured permissions and still produces an undesirable outcome. Those are policy-design questions, and they did not exist in the same form before the technology enabled the behavior.

The partner therefore needs a capability that can be described as Governance Intelligence: the continuous interpretation of the relationship between emerging platform capabilities, business opportunity, new risks, and appropriate controls. That changes the nature of governance services, because a traditional managed governance service focuses heavily on administration, reviewing environments, enforcing policies, monitoring compliance, and remediating exceptions, and while those functions remain necessary they should themselves become progressively automated wherever possible, which moves the premium partner proposition upward.

What changed in the Microsoft ecosystem this quarter? Which newly available capability affects our risk model? Which policies now need revision? Which controls are redundant because the platform provides native enforcement? Which new risks are emerging because agents possess capabilities that traditional applications did not? How should the CoE classify autonomy? When does a design need specialist review? Which agent behaviors require evaluation rather than deterministic test evidence? Governance becomes continuous interpretation rather than static policy maintenance.

The concept matters because the pace of platform evolution makes periodic governance reviews increasingly insufficient: a policy set that was appropriate six months ago can become incomplete once new tool-calling, memory, identity, or agent-to-agent capabilities become available. The CoE needs a mechanism for converting product evolution into governance change, and a strong partner can provide exactly that by combining vendor awareness, enterprise architecture, security, and risk expertise.

The partner should also help the CoE establish a hierarchy of controls, because not every risk should be addressed through manual approval: some controls should be encoded in deterministic services, some enforced through platform policy, some detected through monitoring, and some reserved for human review because the judgment itself is contextual. The value lies in choosing the correct control mechanism rather than in adding more control activity, and as governance execution becomes increasingly automated, governance judgment becomes more valuable.

7. The Innovation Hub - From Abundant Ideas to Industrialized Production

Low-code increased the number of people capable of creating solutions, and generative AI increases the speed at which those solutions can be conceived and prototyped, which together produce an important inversion. Historically many organizations struggled to deliver enough technology against demand, so innovation management focused heavily on increasing delivery capacity; in the agentic era the constraint moves, and the problem becomes an excess of plausible ideas rather than a shortage of them.

Every function can identify dozens of activities that appear suitable for automation or agentification, employees can assemble a prototype in an afternoon, and technology demonstrations make opportunities look compelling long before their economic or operational value has been validated. A conversational proof of concept can create the impression that a process is ready for transformation even when the data, the process ownership, the risk model, or the adoption conditions are immature.

This is where the relationship between the Center of Innovation and the Innovation Hub matters, and where it is frequently drawn in the wrong place. The Center of Innovation does not assess ideas one at a time: it establishes the innovation themes, the criteria, the risk routing, and the evidence a proposal must produce before it can consume investment. The Innovation Hub is the executing arm of that framework, the layer that runs the funnel against those criteria and then carries what survives into delivery, which is why the two halves of the work cannot usefully be discussed apart. Selection and production belong to the same pipeline, and in the agentic era both ends of it become harder.

At the front of that pipeline selection becomes the scarce skill, and the partner has a critical role in it because good selection requires an external perspective. An internal stakeholder may be emotionally invested in a proposed solution, a technology team may be attracted by the novelty of an agentic use case, an executive may have seen a competitor announcement and want an equivalent capability, and a vendor demonstration may create pressure to find a use case for a capability rather than the right capability for an existing problem. A strong partner should be able to challenge all four impulses.

What problem does the idea solve? How frequently does the problem occur? What is the current economic cost? Which users are affected? Will adoption realistically occur? Does the process have sufficient data and operational maturity? Is the proposed autonomy appropriate? Could a simpler solution produce the same value? What happens if the initiative succeeds technically but does not change behavior? What evidence would justify scaling?

Asking those questions well is idea nurture rather than idea collection, and it is work the Hub performs against the criteria the Center of Innovation has set. Many valuable ideas begin in an immature form, representing a genuine opportunity while lacking clarity around value, process, or implementation, and the role of the funnel is not to reject them on sight but to help them mature to the point where a rational

investment decision becomes possible, which is where the partner contributes structured methods, external benchmarks, industry patterns, and experience from comparable transformations.

This is particularly valuable because implementation partners have historically carried a conflict of interest at exactly this point: if downstream delivery generates most of the revenue, more approved initiatives mean more work. The agentic transformation partner has to become comfortable recommending that some initiatives should not proceed, and saying "do not build this" has to become evidence of consulting quality rather than a loss of it.

The partner's success should therefore be linked to portfolio value rather than backlog volume, which requires stronger business cases, clearer hypotheses, explicit success measures, and staged investment, and may also require a culture in which stopping an initiative counts as a successful outcome when the evidence says the value is insufficient. Technology creation becomes abundant while investment attention remains finite, so the funnel is where scarcity has to be deliberately reintroduced.

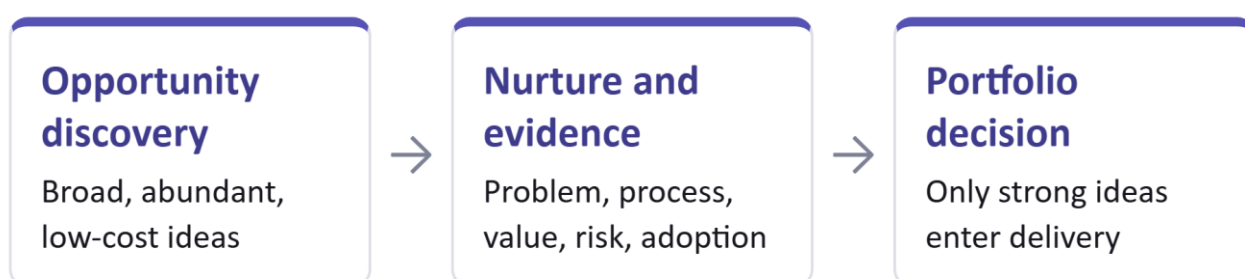


Figure 3. When ideas become abundant, value moves to selection and nurture.

The purpose of the funnel is not to maximize the backlog. It is to improve the quality of what reaches delivery.

Behind that funnel sits the part of the Hub where most partners will feel the transformation most directly, because this is where the bulk of traditional implementation effort resides. A conventional delivery lifecycle consumes substantial human work after an idea is approved. Requirements are elaborated. Data structures are configured. Applications are built. Flows are created. Integrations are implemented. Code is written. Testing is performed. Defects are resolved. Releases are managed. It is precisely this execution layer that AI-assisted development attacks most directly, and the wrong strategic response is to defend it: the partner should not try to preserve manual complexity because that complexity creates billable hours, but should set out to industrialize the factory instead.

That requires a shift in thinking, because the delivery half of the Hub should not be viewed as a collection of developers but as a system that converts a well-designed requirement into a reliable production capability, and if the input is poor the factory will produce poor output faster while if the input is strong automation becomes an advantage. The highest-value work therefore moves toward the beginning, where the process must be correctly understood, the future state designed, the implementation pattern selected, the experience defined, the architectural constraints made clear, the security expectations understood, the

agent behavior specified, and the acceptance and evaluation criteria established. Once that foundation is strong, the factory should become deliberately repetitive.

A successful industrial factory does not derive its competitive advantage from having craftspeople reinvent the production process for every unit; the intelligence resides in product design, machinery, standards, quality systems, and continuous improvement, and production itself becomes predictable. The same should be true here, where a standard class of application should not require an entirely bespoke engineering journey, common integration patterns should be reusable, approved agent architectures repeatable, testing and evaluation frameworks already in place, deployment automated, observability standard, security and governance controls embedded rather than added at the end, and reusable skills and components should reduce duplication. The partner should invest heavily in the system that makes implementation boring.

This may appear counterintuitive for a technology consultancy, since implementation complexity has been proof of expertise for decades, but unnecessary complexity is better read as proof of poor operating-model design, and the premium consulting proposition becomes the ability to make complex transformation decisions while making execution as routine as possible. None of which means that every solution can be produced from a fixed template: enterprise requirements vary, complex scenarios will continue to require bespoke engineering, and industrialization is not the elimination of judgment but the deliberate removal of repeatable work from the places where judgment is not required.

A mature Hub therefore distinguishes between design variability and production variability, since design may vary because business context varies while production should vary only where the approved design genuinely requires it, and the partner's reusable assets - architecture patterns, components, skills, APIs, evaluation sets, and deployment pipelines - exist to reduce the unnecessary kind.

This is also where solution selection becomes important, because the partner should not assume that every problem requires the same technology pattern: a deterministic process may be better solved with Power Automate, a structured operational process may need a model-driven application, a high-frequency conversational support scenario may justify Copilot Studio, and a knowledge task within Microsoft 365 may fit a different agent pattern altogether. The Hub needs experts who can select the simplest pattern that satisfies the requirement rather than maximizing the sophistication of the technology, and that judgment does not improve merely because generation has become cheaper.

Across both halves of the Hub the movement is the same. Value shifts from choosing well by instinct and building everything by hand toward designing the conditions under which selection and construction both become reliable and repeatable.

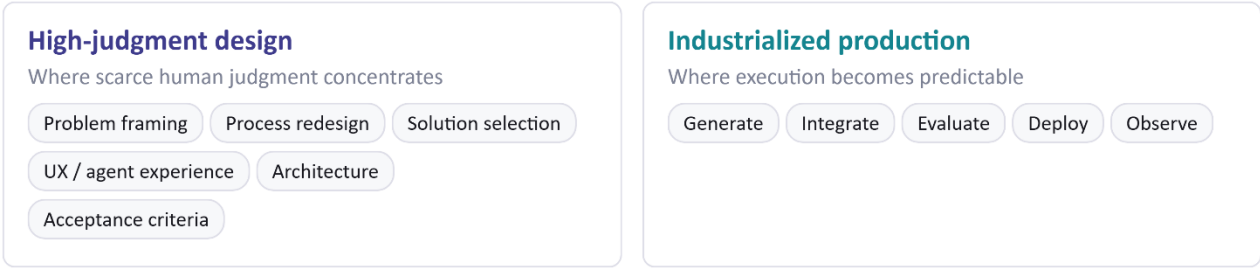


Figure 4. Intelligence before the factory.

The Innovation Hub should concentrate scarce human judgment before the production line and use reusable digital capability to make execution predictable.

8. Supervised Design - The Discipline Between Consulting and Industrialized Delivery

If the factory becomes more automated, someone must define its inputs and determine whether its outputs are acceptable, and that is the purpose of Supervised Design. It is not simply the use of AI to accelerate existing development practices but a redistribution of human expertise across the lifecycle: where a consultant in the traditional model spent substantial time producing the artifact itself, in a supervised model more of that time moves toward defining intent, constraints, and quality.

Consider a simple application, which a low-code maker would once have built by hand as tables, forms, screens, flows, and rules, and whose initial structure an increasingly generative experience can now produce from a natural-language description. The human challenge changes accordingly. Was the process described correctly? Does the proposed data model represent the business accurately? Are the user roles appropriate? Which edge cases were omitted? Which data should not be exposed? What should happen when the process fails? Does the design comply with enterprise architecture? Is the generated experience actually usable? The machine can accelerate artifact creation, but it cannot eliminate the need for a high-quality definition of the problem.

The same principle becomes more important with agents, because agent quality cannot always be assessed through deterministic output alone: an agent may answer the same underlying request in different ways, choose different tools depending on context, need to recognize when it lacks sufficient information, and need to escalate rather than act, so quality has to be expressed through behavioral expectations and evaluation criteria rather than through a single expected result.

The design discipline must therefore answer questions such as what the agent is trying to achieve, which information sources are authoritative, which tools it may use, what it should never do, when it should ask for approval, how it should communicate uncertainty, what constitutes an acceptable outcome, how failure will be detected, and how behavior will be monitored over time. Those are not prompts in the narrow sense; they are system design.

This is why Supervised Design requires several disciplines working together: process consultants define the work, business analysts clarify the problem, functional consultants translate the process into platform patterns, UX and agent experience designers define the interaction, architects define boundaries and dependencies, developers establish engineering capabilities and validate generated artifacts, and governance specialists define controls, after which AI operates inside the design envelope those disciplines have established.

The model also changes the meaning of quality assurance, since traditional testing verifies whether a predefined sequence produces the expected result while agentic evaluation must additionally assess whether the system selected an appropriate capability, used authoritative evidence, respected boundaries, communicated uncertainty, and behaved consistently across representative scenarios. The partner

therefore needs evaluation design as a core delivery capability rather than as an afterthought performed at the end.

Supervised Design also helps prevent a common failure mode of generative technology, which is producing a superficially plausible artifact that encodes a weak assumption. When generation is fast, errors amplify quickly: a bad data model propagates across multiple screens and flows, a weak agent instruction affects every interaction, and a poorly bounded tool creates risk at scale. The design envelope exists to catch those things before they are industrialized.

Its strategic importance is that it allows partners to embrace implementation automation without surrendering quality, and it creates a new measure of professional value, because the expert is no longer valuable for having manually touched every artifact but for the fact that the artifacts produced under their supervision are consistently fit for purpose.



Figure 5. The Supervised Design lifecycle.

Human expertise dominates intent, constraints, and evaluation; AI progressively expands execution capacity inside the resulting design envelope.

9. Center of Enablement - Sustainable Adoption Is Not a Training Schedule

Enablement is another capability where traditional service models can confuse effort with outcome, since a partner may deliver hundreds of training sessions, publish extensive documentation, and run large communities while adoption remains superficial. Attendance is not adoption and completing a course is not capability; the relevant question is whether people can use the platform appropriately and repeatedly in the context of real work.

Agentic technology makes the distinction sharper, because different populations need fundamentally different forms of enablement. A maker needs to understand where AI-assisted generation is useful and where manual design remains necessary, a business analyst needs to know how to use AI for analysis without delegating problem framing to it, a process owner needs to understand which work can be agentified, a manager needs to understand how to supervise teams in which some execution is performed by agents, an architect needs to understand agent boundaries, tools, security, and observability, and a business user needs to recognize when an agent output requires verification and when autonomous execution is permitted. None of that is solved by a generic Copilot Studio training course.

The Center of Enablement therefore needs to design learning as an organizational system, in which some capability is developed through formal training, some comes from communities, some from hands-on experimentation, some requires coaching, and some should be embedded directly into the flow of work. AI itself can become part of that model: a maker assistant can explain approved patterns contextually, an architecture assistant can guide teams toward enterprise standards, a governance agent can explain why a particular action is restricted instead of simply blocking it, and a learning agent can recommend resources based on role, maturity, and observed behavior.

This changes the partner proposition, because rather than selling trainers the partner designs an enablement architecture, asking what capabilities each persona requires, which behaviors indicate maturity, where human coaching should remain, which questions can be answered by contextual agents, what community mechanisms create peer learning, and how enablement data should inform future interventions. The partner then measures sustainable adoption rather than activity volume: are users becoming more autonomous, are the same mistakes decreasing, are approved patterns reused, are governance exceptions falling, are makers progressing in maturity, and is the organization producing higher-value solutions rather than merely more of them?

The agentic era also changes what should be taught, since traditional platform enablement focuses on how to use features while future enablement must increasingly include judgment: when a user should trust an agent result, when a maker should use a deterministic workflow rather than agentic reasoning, how a process owner should define human approval thresholds, and what responsibility a manager retains when an agent performs the work. Those are operating-model skills rather than product skills.

A strong partner can also help the CoE create role-based learning paths that evolve as the technology does, because the curriculum for an agent maker in 2026 will not stay stable while new orchestration, memory, identity, tool, and evaluation capabilities continuously change the design space, which means enablement needs the same adaptive quality as governance.

Enablement is also where the internal community sits, because the community is the surface on which enablement is actually experienced: it is where makers, champions, and business owners meet the CoE, discover what already exists, and find the route to help. For the partner it is rarely a source of implementation revenue, which is precisely why it is under-served, and designing a community that sustains itself is a capability question rather than an activity schedule, belonging in the same conversation as the enablement architecture rather than after it.

If the customer still requires the partner to answer every routine question several years into a CoE program, the partner may have created revenue but failed to create maturity, whereas the stronger long-term relationship is one in which the customer's internal capability improves while the partner continuously brings new expertise, external perspective, and institutional knowledge. That has commercial consequences beyond enablement, and Chapter 13 returns to it.

10. Sustain - From Brute Force to Agentified Operations

The Sustain layer may be the clearest demonstration of why partner economics need to change, because traditional managed services scale approximately with the size and complexity of the estate: more applications mean more support, more environments mean more administration, more users mean more incidents and requests, and more components mean more monitoring and maintenance. That produces a predictable commercial proposition in which the support team grows as the platform grows.

The agentic CoE should challenge that relationship, since growth in platform value should not require proportional growth in human operations, which means the partner has to approach Sustain as a process-transformation problem. The first step is not to introduce agents but to understand demand, and to ask why each category of operational work exists at all.

Some demand should be eliminated, because a recurring incident may indicate that an application needs redesign, a repeated access request may reveal poor identity design, a frequent user question may indicate weak product experience, and a maintenance activity may exist only because an older implementation lacks automation. The best support ticket is often the one that never needs to be created.

A second category should be automated deterministically, since an operational activity that follows a clear rule is usually more reliable, cheaper, and easier to govern as conventional automation than as an agent, which is where password resets, scheduled housekeeping, standard notification patterns, and simple environment operations tend to belong, depending on the organization's architecture.

A third category is genuinely suitable for agentification, covering activities that require interpretation, retrieval of contextual information, correlation across sources, or multi-step interaction with tools, where an agent can gather evidence, classify the issue, perform safe diagnostics, propose remediation, and escalate when its confidence is insufficient. Only the final category should routinely reach specialist humans: complex exceptions, problem management, risk decisions, architecture improvement, and activities requiring explicit accountability.

This produces a four-part operating principle:

OPERATING PRINCIPLE

Eliminate what should not exist. Automate what is deterministic. Agentify what requires bounded interpretation. Elevate humans to the work where judgment matters.

That is considerably more powerful than using AI in support, because it redesigns the operational model rather than accelerating it, and it should be applied not only to incidents but to Platform Operations & Administration, where environment provisioning, policy checks, capacity analysis, solution-health monitoring, ownership reviews, lifecycle tasks, compliance evidence, and platform communications all contain opportunities for elimination, deterministic automation, or agentic assistance. The partner should help the CoE map those activities and deliberately reduce the amount of routine human intervention.

The commercial consequences matter just as much, because a managed service priced mainly around ticket volume is misaligned by construction: if the partner reduces the number of tickets, it reduces its own revenue. A capability-oriented managed service needs different measures, among them platform health, reduction in preventable incidents, the percentage of demand resolved without human intervention, mean time to resolution, automation and agentification coverage, user experience, reduction in recurring root causes, and continuous improvement delivered, and under those the partner benefits when Sustain becomes more efficient, which is far better aligned with what the customer actually wants.

The partner also needs to recognize that agentic operations introduce their own operational burden, since agents require observability, evaluation, knowledge maintenance, tool lifecycle management, cost monitoring, and governance. Agentification is not a free replacement for human effort but an operating-model investment, and the economic case depends on whether the total system becomes more scalable and reliable rather than on whether a ticket can be answered by a chatbot.

The future Sustain proposition should therefore be explicit: we do not provide the people required to operate today's model indefinitely; we continuously redesign the model so that fewer activities require those people.

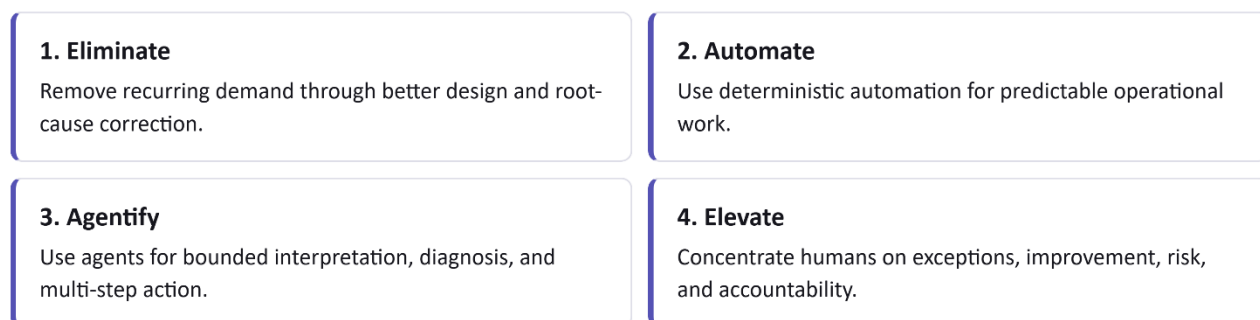


Figure 6. The Sustain transformation model.

Sustain should be designed to reduce human operational demand before the partner adds more capacity.

Part III: Rethinking Talent

11. Human Value Moves Toward Ambiguity, Judgment, and Accountability

The argument that AI changes the partner talent model can easily become simplistic: one common version says developers will be replaced by consultants, another says all technical roles will simply become more productive, and neither captures the full transition.

The better principle is that human expertise moves toward the areas where the work is hardest to encode and where accountability remains consequential, so that as agents take on more execution, humans increasingly contribute by setting intent, applying critical judgment, defining quality, and owning outcomes, which changes the relative importance of several disciplines inside partner organizations.

11.1 Process consultants - designing work before technology

Process consultants should become one of the defining roles of the agentic Power Platform partner, because traditional automation frequently begins with an existing process map and asks which steps can be digitized, whereas agentic transformation requires a more fundamental question: should this process still exist in its current form?

Many enterprise processes are artifacts of historical constraints, where a hand-off exists because two legacy systems could not share data, a manual review exists because information had to be assembled from multiple sources, a coordination role exists because no system possessed sufficient context to manage the workflow, and a customer had to complete a structured form because unstructured input could not be interpreted reliably. When those constraints change, automating the historical process merely preserves unnecessary work.

The process consultant therefore analyzes purpose before sequence, asking what outcome the process is designed to create, where value enters, which activities exist only because of old technology, which decisions require accountability, which tasks require human empathy or negotiation, which can be delegated safely, where agents can assemble information before a human decision, and where deterministic automation can replace an approval step entirely. This is human-agent operating-model design, and the partner that develops the capability moves upstream into the transformation conversation, while the partner that does not risks becoming the recipient of somebody else's process design, competing mainly for implementation.

11.2 Business analysts - problem framing becomes more valuable as documentation becomes easier

AI is already very capable at the activities traditionally associated with business analysis, summarizing workshops, organizing requirements, drafting user stories, creating acceptance criteria, comparing

documents, and producing process descriptions, so if the business analyst's value lies primarily in transforming one form of documentation into another, that value is vulnerable.

The more defensible role is problem framing, because stakeholders rarely present perfectly defined problems and instead present requests shaped by their own experience: "We need another application", "We need to automate this approval", "We want an agent", "We need a dashboard". A strong analyst treats those statements as hypotheses rather than requirements, asking why the application is necessary, why the approval exists, what decision the agent would support, and what behavior should change because the dashboard exists, then investigating the underlying condition, quantifying impact, identifying affected personas, distinguishing symptoms from causes, and determining what evidence would demonstrate that the problem has been solved.

The discipline becomes more important as technology gets cheaper to produce, because when implementation is expensive poor ideas often die naturally, since nobody can justify the cost, whereas when implementation becomes easy an organization can create low-value technology at extraordinary speed, which makes problem framing a quality gate for innovation.

11.3 Functional consultants - from knowing the product to knowing the right pattern

Functional consultants have historically differentiated through detailed platform knowledge, understanding how the product behaves, which configuration options exist, where the limitations are, and what the implications of different settings might be. That expertise remains valuable, but generative assistants increasingly make basic product knowledge easy to reach, so the premium moves from knowing how to configure the product toward knowing which configuration represents the right solution.

A Copilot may be able to explain how to configure an approval workflow, but it cannot determine whether an approval workflow is the correct operating-model choice for a particular organization; a generative tool may create a Dataverse structure, and the functional consultant must recognize whether the data model reflects the business correctly; an agent builder may expose a tool, and the consultant must understand whether that tool belongs inside the agent's responsibility boundary.

The future functional consultant is therefore a pattern expert who combines understanding of the platform with understanding of business capabilities and architectural consequences, which places the role at the center of Supervised Design.

11.4 UX and agent experience designers - designing trust as well as interaction

Low-code democratization sometimes encouraged a view that functional applications did not require dedicated design capability, on the argument that a screen created quickly could always be refined later. Agentic systems make that attitude increasingly risky, because the experience is no longer limited to screens: users may interact with a system conversationally, and an agent may act proactively, execute work in the background, require confirmation, hand work back to a person, or coordinate multiple systems without exposing the underlying steps.

This changes the design problem from interface design to relationship design, and the questions that follow are questions about trust. How does the user understand what the agent is doing? How does the system communicate uncertainty? When must the agent explain its reasoning? How can users correct it? How does the human regain control? When should the interaction shift from conversation to a structured interface? Which actions require explicit confirmation? How should failure be communicated without destroying trust?

Those decisions directly affect adoption, because an agent that is technically capable but behaviorally confusing will fail, and the partner therefore needs designers who understand both conventional UX and agent behavior, which makes conversation design part of a broader discipline of Agent Experience Design.

11.5 Architects - coherence becomes scarcer as creation becomes easier

Architecture often becomes more important during periods of technological democratization, because architectural consistency can be maintained through direct control while only a small central team creates applications, and becomes impossible to maintain that way once thousands of makers and agents can create technology, at which point the organization needs patterns instead.

Power Platform already demonstrated the effect with citizen development, and agentic technology amplifies it: an enterprise may have hundreds or thousands of agents performing different functions, some using shared tools, some invoking APIs, some accessing the same knowledge, some operating inside Microsoft 365, and some embedded in line-of-business processes, so that without strong architectural principles the resulting estate becomes more fragmented than the application landscape it was meant to simplify.

Architects therefore need to define responsibility boundaries. When should a capability be a reusable tool rather than embedded logic? When should multiple agents share a skill? Where should data be stored? How should identity work? How should agent-to-agent interaction be governed? Where should MCP be used to expose a capability to many agents rather than wiring it into one? What belongs inside Power Platform rather than Azure or another enterprise platform? How should observability work? How should costs be controlled? Those are system-level questions, and the architect's value shifts accordingly, from designing individual solutions toward designing an ecosystem in which many solutions can emerge coherently.

11.6 Developers - from producing artifacts to engineering reusable capability

Developers remain indispensable, since enterprise systems still require secure integrations, complex extensions, performance engineering, data engineering, APIs, observability, deployment automation, and specialist user experiences, and agents increase the need for many of those capabilities because agents require tools through which to act. The shift is not from developer to no developer but from repetitive development toward leveraged engineering.

A developer who manually implements the same pattern ten times creates ten units of delivery, whereas a developer who creates a reusable capability that ten teams or agents can consume creates leverage, which is the role of the Capability Engineer. Capability Engineers build connectors, expose APIs, create MCP servers, create reusable components, establish security frameworks, create evaluation infrastructure, develop observability tooling, build agent tools, implement deployment pipelines, and codify engineering standards into automated checks, so that their work becomes part of the organization's production system.

That is a more strategic use of engineering talent, and it introduces a new challenge, because junior developers traditionally acquired expertise by performing repetitive implementation under supervision, and if agents now perform more of that repetitive work then partners must redesign how engineering judgment is developed: junior professionals still need to understand why generated code is good or bad, and they still need exposure to architecture, security, debugging, failure analysis, and performance. Partners therefore need deliberate apprenticeship models that use AI without allowing it to remove the learning experiences required to create future senior engineers.

What that looks like in practice is narrower than it sounds. Junior engineers no longer learn by producing the tenth variant of a pattern, so the learning has to move to where judgment is now exercised: reviewing generated output against a standard and being required to say why it fails, sitting in the design conversations that set the envelope rather than receiving its output, owning the evaluation set for a capability rather than the capability itself, and debugging failures in production systems they did not build, all of which are supervisable and none of which depends on repetitive production. What they demand instead is senior time, which is the scarce resource the same transformation is consuming, and which is why apprenticeship has to be designed and funded rather than expected.

11.7 Governance specialists - from administering controls to designing them

Governance has historically been staffed with people who know today's controls: which policy blocks which connector, how an environment strategy is enforced, what a data policy covers. That knowledge remains necessary, and it is also the part of the discipline most exposed to automation, because platform administration is precisely what vendors improve.

The governance specialist of the agentic era is closer to a risk analyst than to an administrator, since the work is to recognize that a newly available capability has created an exposure the existing policy set does not describe, and then to decide which control mechanism answers it: a platform policy, a deterministic service, a monitoring signal, or a human review that cannot be delegated because the judgment is contextual. That is the Governance Intelligence described in Chapter 6, expressed as a role rather than as a service.

It is also a different hiring profile, requiring enough architecture to understand what an agent can reach, enough security to know why that matters, enough business context to judge consequence rather than severity, and enough vendor awareness to see a change coming before a customer meets it in production. Partners that staff governance exclusively from platform administration will find the administration automated and the judgment absent.

11.8 Industry experts - the knowledge that does not arrive with the platform

Product knowledge is portable and increasingly accessible, while domain knowledge is neither, because a consultant who has been through a dozen banking, aviation, retail, or manufacturing transformations knows which processes look automatable and are not, where regulation actually binds, which exceptions consume the operational cost, and which improvement the organization has already attempted twice.

That expertise becomes more valuable as implementation compresses, since the questions that remain are questions about the business, and it is also the most defensible asset a partner holds, because it cannot be acquired from documentation and does not transfer with a certification.

The implication for talent strategy is uncomfortable. Industry experts are usually the least visible people in a delivery organization, billed against roles named after the platform and recorded as advice given inside somebody else's workstream, so a partner that intends to compete on domain knowledge has to name the role, hire for it, and give it a career path that does not require becoming a platform specialist first. It is also the profile whose knowledge most repays being encoded, which is the subject of Chapter 12.

11.9 The delivery pyramid is not simply smaller - it is rebalanced

The traditional partner delivery pyramid places a relatively small number of senior consultants and architects above a much larger execution layer, which works while implementation is the principal bottleneck. Agentic delivery introduces a different form of leverage, because more execution capacity can be provided digitally, and the human organization therefore shifts toward the areas requiring design, judgment, and accountability.

None of which should be read as a universal prediction that every project will use fewer people, since transformation programs can expand scope as productivity increases and entire categories of work that were previously unjustifiable may become feasible. The more defensible claim is that the relative composition of teams will change, with more process expertise, more architecture, more experience design, more governance, more evaluation, and more senior engineering leverage, and with less value attached to repetitive manual production.

That rebalancing carries strategic implications for recruitment, career progression, and acquisitions, because a partner that continues to optimize its talent strategy around historical delivery ratios may discover that it holds too much of the work being compressed and too little of the work becoming scarce.

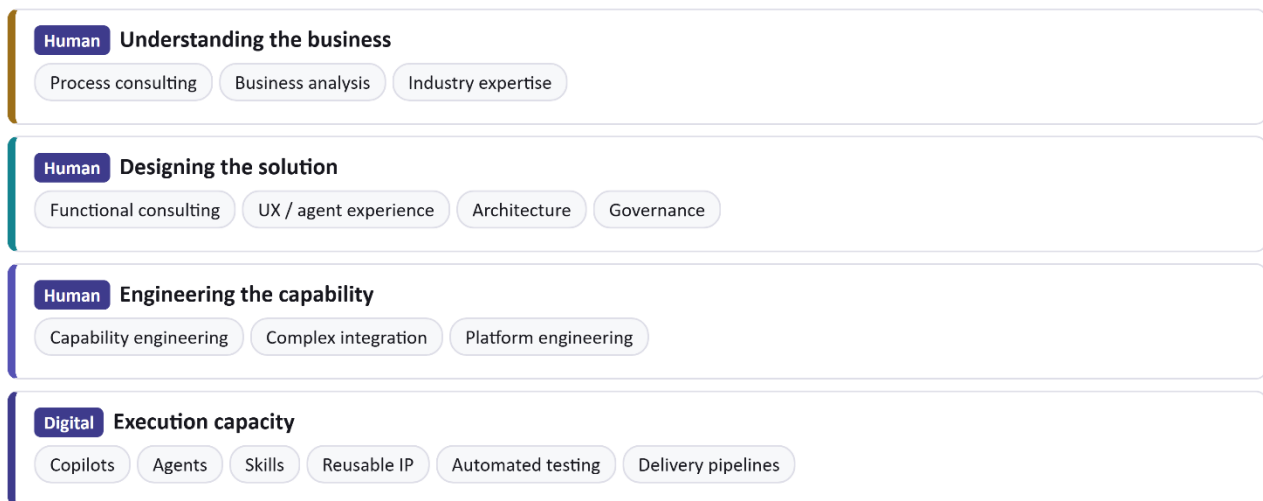


Figure 7. The rebalanced partner delivery model.

The change is not a simple reduction in people. Human scarcity moves toward ambiguity, judgment, and accountability while digital execution capacity expands.

Part IV: Rethinking Consulting

12. The Return of Consulting and the Rise of Executable Consulting IP

There is an important historical irony in this transformation, because some of the capabilities that become more valuable in the agentic era resemble the capabilities that distinguished strong enterprise consultancies before large-scale delivery factories became dominant.

Experienced Dynamics consultants understand this well. A CRM transformation was rarely valuable because somebody knew how to create a table or configure a form; those skills were necessary, but the transformation happened when the consultancy helped the organization decide how sales should operate, how customer relationships should be managed, which information should be available, how teams should collaborate, and how performance should be measured, with technology implementing the operating model that resulted.

Over time, as implementation demand expanded, parts of the partner ecosystem became increasingly delivery-centric, and there was strong commercial logic behind that evolution, since customers had enormous backlogs and partners could scale revenue by scaling implementation capacity.

The agentic era makes the earlier consulting capability important again, because when customers can build more themselves the partner has to contribute knowledge that is harder to obtain from the platform: cross-customer pattern recognition, industry insight, operating-model design, transformation judgment, architecture, governance intelligence, experience, and the ability to challenge executive assumptions. The partner does not therefore become less technical; it combines technology expertise with a stronger consulting identity.

That move creates its own economic challenge, because senior consultants are expensive and scarce, and a firm that simply replaces a large delivery pyramid with a large population of senior advisors has not changed the relationship between growth and headcount at all. Agentic technology offers an alternative, in which the partner increasingly encodes portions of its institutional knowledge into reusable digital capabilities, and that is the concept of Executable Consulting IP.

Traditional consulting firms already possess substantial intellectual property in the form of methods, assessment frameworks, reference architectures, checklists, industry process models, governance frameworks, lessons learned, accelerators, and training materials, but most of it is passive: a consultant reads it and applies it. Agentic systems allow parts of that knowledge to become active, so that a governance framework can become an agent that guides classification, an architecture standard can become an automated review capability, a discovery methodology can become an interview and analysis agent, an industry process model can help consultants identify likely automation opportunities, a design standard can become an evaluation tool, and a support playbook can become an operational diagnostic capability.

The direction of the platform matters here, because as agent behavior becomes expressible in modular, reusable sets of task-specific instructions that can be shared across agents, expertise can increasingly be packaged as behavior rather than as documentation. The strongest consultant still creates the insight, but the organization no longer depends solely on that individual's availability to apply it.

That produces several benefits at once: delivery quality becomes more consistent, junior consultants gain access to stronger guidance, the partner can scale differentiated knowledge without proportional headcount growth, and the intellectual property becomes harder to copy, because it incorporates the firm's accumulated experience rather than only public product knowledge. Learning compounds as well, since each engagement can improve the reusable capability. The strategic question for partner leadership therefore becomes: How much of what our best people know exists only in their heads? Every answer represents both a risk and an opportunity.

Executable Consulting IP also changes how partners should think about accelerators, because traditional accelerators take the form of prebuilt solutions or code, whereas in the agentic era an accelerator can be knowledge, instructions, evaluation sets, agent patterns, process models, and decision frameworks, which do not need to replace the platform but can shape how it is used.

This is particularly powerful in industry-specific consulting, where public platform knowledge is easy for competitors to acquire while deep domain knowledge accumulated across dozens of banking, aviation, retail, or manufacturing transformations is much harder to reproduce, so that encoding it into reusable discovery agents, risk classifiers, solution patterns, and evaluation frameworks creates a form of institutional leverage that goes beyond headcount.

Encoded IP is an asset with an operating cost, and this is where most reuse programs fail rather than at creation. A skill that encodes a governance framework is a living asset in exactly the sense I use for customer-facing agents in Copilot Studio and the Future of the Power Platform Center of Excellence: it has an owner, a review cycle, and a decay rate. When the platform ships a capability the framework did not anticipate, the encoded version is quietly wrong while remaining available, which is the same silent degradation the CoE is told to watch for. Compounding is therefore not automatic, and it happens only where each engagement is expected to return improvements to a named owner, where a capability nobody has reviewed within a defined period is withdrawn rather than left in the catalog, and where the cost of maintaining the catalog is carried by the firm rather than by whichever project last touched it.



Figure 8. Executable Consulting IP.

The goal is not to remove senior consultants, but to allow their judgment to influence far more work than their individual calendars permit.

13. Commercial Models Need to Reward Leverage Rather Than Effort

A partner cannot fully transform if its commercial model punishes productivity, and this is one of the hardest aspects of the transition, because the economics of professional services are deeply embedded in forecasting, sales incentives, and utilization models.

Time and Materials remains useful, since there will always be uncertain work and customers will continue to request capacity, but the model creates a structural tension once AI reduces effort: if an activity takes half as long, the customer reasonably expects the cost attached to that effort to fall. Fixed-price models let partners capture more of the productivity gain, because delivering the same outcome with less effort improves margin, yet competitive markets eventually absorb those improvements too, and when every credible partner can deliver faster, price expectations adjust.

The long-term response is to move toward propositions whose value is associated more closely with capability and outcome: a process redesign has value because it changes the economics of work, an agentic governance framework has value because it enables safe innovation, an Innovation Hub redesign has value because it improves throughput and reuse, and a Sustain agentification program has value because it reduces recurring operational cost. Those can still be priced using conventional models, but the price conversation begins with the outcome rather than with the number of people supplied.

The service portfolio should therefore become more deliberate, with propositions that might include process transformation assessments, agent opportunity portfolios, CoE evolution roadmaps, agentic governance frameworks, Innovation Hub transformation, Supervised Design services, experience architecture, agent portfolio architecture, Sustain agentification programs, enablement strategy, continuous governance intelligence, and continuous agent optimization. Implementation remains part of many of those offers; it simply stops defining them.

Managed services also need to evolve, because a support model that earns more revenue when more incidents occur is poorly aligned with an agentic Sustain objective. A more mature proposition prices against the Sustain measures set out in Chapter 10 - platform health, preventable incidents, demand resolved without human intervention, automation and agentification coverage, recurring root causes eliminated - with continuous agent evaluation added, because an agentified service that is not evaluated is not a service. Under that model the partner is financially rewarded for making the customer more efficient, which is the commercial expression of capability augmentation.

The same principle should influence sales incentives, because salespeople rewarded almost exclusively for selling large teams and long projects will naturally resist propositions that use fewer people and more reusable IP, and business units measured primarily by headcount and utilization will treat leverage as a threat rather than as an advantage. Commercial transformation therefore requires internal incentive transformation.

Partners also need to become comfortable with a different relationship between price and effort, since a highly reusable assessment capability may require very little delivery time while creating substantial value, and pricing it only by consultant hours undervalues the intellectual property that made the efficiency possible. Partners need commercial mechanisms that recognize the value of methods, data, agentic assets, and reusable expertise.

None of this requires an immediate move to pure outcome pricing, which can be difficult because partners do not control every variable affecting business results. The more practical objective is to reduce the automatic equation between price and human effort, and to increase the proportion of the proposition justified by capability, risk reduction, speed, reusable IP, and measurable improvement.

Create capability, not dependency

A mature Center of Excellence is intended to increase organizational capability, which creates a tension for traditional consulting, because some service models unintentionally benefit from dependency: the customer cannot operate the solution without the partner, knowledge remains external, architecture decisions are poorly documented, and support becomes permanent because internal teams never become capable. The model may generate recurring revenue, but it conflicts with the purpose of a mature CoE.

The future partner needs a more confident proposition, one that enables the customer, transfers knowledge, creates reusable assets, trains internal specialists, helps establish communities, builds governance capabilities, and creates executable IP the customer's CoE can use, so that the customer becomes stronger because of the engagement.

The partner remains relevant because it continues to bring new insight rather than because it has preserved a knowledge asymmetry, and a customer with a mature CoE may need less routine implementation support while valuing the partner more for external perspective, industry benchmarking, architecture review, emerging technology insight, governance intelligence, complex engineering, strategic transformation, and intellectual property. Dependency is replaced by trust, which is a stronger long-term position.

Part V: The Partner Transformation Agenda

14. Partners Need to Become Customer Zero of Their Own Operating Model

Partners will struggle to credibly advise customers on agentic operating models if their own professional-services processes remain largely unchanged, and using Microsoft 365 Copilot internally is not enough: the partner needs to examine its own value chain with the same discipline it proposes for customers.

Consider presales. How much effort is spent searching previous proposals? How much knowledge remains locked inside individual consultants? How much estimation is repeated manually? Could an agent assemble relevant precedent while a solution architect retains responsibility for the commercial judgment?

Consider delivery. Which documents are created because the methodology genuinely requires them, and which exist because they always have? Where can generation reduce administrative effort? Which quality controls can be encoded? Which architectural reviews can be partially automated? Which recurring implementation activities should become reusable platform capabilities?

Consider support. Which incidents recur? Which diagnostic activities can become agentic? Where can self-service eliminate demand? Which operational procedures exist simply because internal systems are fragmented?

Consider talent development. Are junior employees learning by performing work that AI will increasingly perform, and if so, how will the organization create alternative learning experiences that develop judgment rather than task familiarity?

That internal transformation creates practical experience, because the partner learns where agents actually help and where they create additional risk, which organizational incentives resist change, which professional roles need redesign, and what it is like to face the same governance questions it will meet in its customers. The strongest agentic consulting methodologies are likely to emerge from the partners that have already transformed themselves.

14.1 A CoE-centered transformation agenda for partner leadership

Partner transformation becomes easier to operationalize when leadership uses the CoE capability map as a diagnostic tool.

Strategy & Advisory. Leadership should ask whether the firm participates sufficiently early in customer decision-making, because if most engagements begin after the customer has already defined a technical requirement then the partner remains downstream, and investment should move toward industry knowledge, process consulting, business architecture, and value assessment.

Center of Governance. The question is whether the partner is building a repeatable capability for interpreting technology evolution, since a future governance practice should not depend exclusively on

administrators who know today's controls but should contain specialists who can continuously connect new product functionality to risk, policy, and operating-model implications.

Center of Innovation. The partner should assess whether its own commercial incentives allow it to argue for better criteria rather than for more volume, which comes down to a single question: can consultants recommend stopping an initiative even when implementation revenue would have followed? If not, advisory credibility will remain limited whatever the criteria say.

Innovation Hub. Leadership should measure both ends of the pipeline. At the front, how much demand is shaped, and how much is stopped before it consumes delivery? At the back, how much delivery is reused, how much testing is automated, how consistently are standard architectures applied, and how much implementation still depends on bespoke manual effort? Every recurring pattern should be treated as a candidate for reusable capability.

Center of Enablement. The question is whether partner activity results in sustainable customer capability, which means training volume should not be the dominant measure and the partner should be able to demonstrate changes in behavior, maturity, reuse, and autonomy.

Sustain. The partner should measure reduction in operational dependency: are incidents being eliminated, is automation increasing, are agents resolving appropriate classes of demand, and are human specialists moving toward problem management and continuous improvement?

This framework allows partner leadership to make the transformation tangible rather than discussing AI strategy abstractly.

15. Three Stages of Partner Evolution

Partner transformation will not occur in a single step, and a useful maturity model contains three broad stages.

The first is the Capacity Partner, an organization that remains primarily oriented around human delivery, where AI may be used by individual employees while the service portfolio and the economics stay largely unchanged, growth remains closely correlated with billable headcount, development and support dominate, intellectual property is mainly static, and governance and enablement services remain activity-based. There is nothing inherently unsuccessful about that model today, and many such firms will continue to grow in the short term because market demand remains strong; its vulnerability is structural rather than immediate.

The second stage is the AI-Enabled Implementation Partner, a firm that has integrated AI into delivery, where developers work with coding agents, documents are generated faster, testing is increasingly automated, internal agents accelerate presales and knowledge retrieval, delivery margins improve, and some reusable agentic IP appears. That is meaningful progress, and it is also the stage at which many organizations mistakenly believe the transformation is complete, because the underlying proposition may still be identical: the partner sells implementation effort and merely executes it more efficiently.

The third stage is the CoE Transformation Partner, an organization that differentiates through its ability to improve the customer's operating model. It has meaningful Strategy & Advisory capability, it understands processes and industries, it provides Governance Intelligence, it helps the Center of Innovation set better criteria and the Innovation Hub apply them, its delivery is built around Supervised Design and industrialized production, its Enablement services create customer autonomy, its Sustain services actively reduce manual operational demand, its engineering talent builds reusable capabilities, its consulting knowledge increasingly exists as Executable Consulting IP, and its commercial model benefits from leverage rather than fearing it. At that stage AI is not simply a technology in the portfolio; it has changed the partner's identity.

The stage a firm occupies is best read from the diagnostic in the previous chapter rather than from ambition. Answer the six capability questions honestly. A firm whose answers are mostly about activity - sessions delivered, tickets handled, administrators supplied, developers staffed - is a Capacity Partner regardless of how much AI it uses internally, a firm whose answers describe the same activities performed faster is at the second stage, and only a firm whose answers describe changes in the customer's operating model, evidenced by measures the customer also recognizes, has reached the third. The transition between stages is not a program; it is the point at which the answers change.

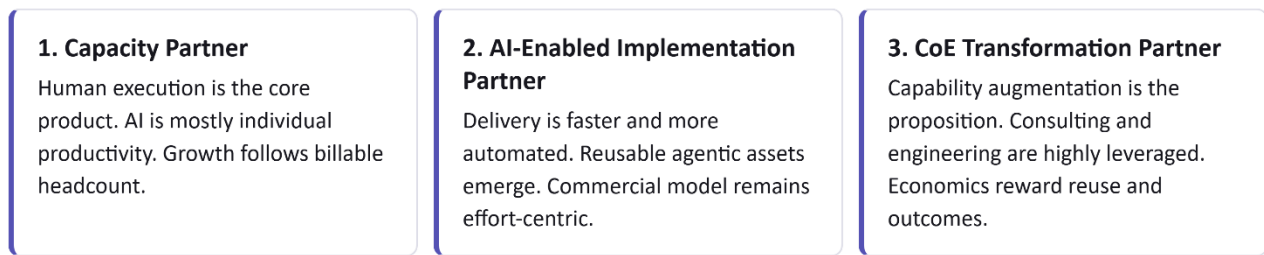


Figure 9. Three stages of partner evolution.

AI-enabled implementation is an important transition stage, but it is not the same as business-model transformation.

16. Metrics Will Reveal Whether the Partner Has Really Changed

Organizations become what they measure, and what goes unmeasured goes unimproved, so a partner that continues to optimize almost exclusively for billable utilization will naturally resist anything that reduces billable effort, a managed service rewarded for ticket volume has little incentive to eliminate tickets, and an Innovation Hub that measures output through the number of applications delivered will optimize production rather than value. The agentic partner therefore requires a broader measurement system.

Utilization will not disappear, because professional-services organizations still need to manage capacity, but it should sit alongside measures of leverage and outcome. How much reusable intellectual property is being created? How many engagements consume those reusable capabilities? How much manual effort has been eliminated from comparable delivery patterns? How much operational demand has been automated or agentified? How much adoption persists after formal enablement ends? What percentage of innovation ideas are stopped before unnecessary implementation? How quickly does a qualified idea reach measurable value? How autonomous has the customer become?

One of those is worth defining fully, because a measure that cannot be computed will not be used. Reuse consumption: for each engagement, record which reusable capabilities it consumed from the firm's catalog, and express the total as the share of delivered engagements that consumed at least one. It needs a catalog with identifiers, one field on the engagement record, and no new system, and its baseline in most firms today is close to zero, which is what makes the first year of movement legible. Each of the other measures needs the same treatment before it is adopted.

Metrics like those reward a different organization, one that makes a support team proud of reducing tickets rather than processing more of them, makes an engineer proud when one API eliminates ten custom integrations, makes a consultant proud when a weak project is stopped before it consumes investment, and makes the CoE proud when growth in business value no longer requires proportional growth in headcount.

The partner also needs measures that reflect the quality of Supervised Design, because reuse without quality simply scales mistakes and productivity without evaluation accelerates defects, so industrialization should be paired with metrics around architecture compliance, automated evaluation coverage, escaped defects, agent behavioral quality, security exceptions, and lifecycle health.

Commercial metrics should evolve as well. What proportion of revenue comes from advisory, intellectual property, and continuous optimization? How much margin improvement results from reuse rather than from rate increases? How much revenue is recurring because the partner provides evolving insight rather than because the customer remains operationally dependent? Those questions reveal whether the firm has genuinely changed its economic model or has merely adopted new technology inside the old one.

17. The Transition Should Begin Before the Old Model Stops Working

The traditional implementation business is not about to disappear, and the point matters because otherwise the argument becomes unnecessarily alarmist: enterprise application estates remain complex, organizations still have enormous backlogs, migration programs continue, security and integration remain difficult, many customers lack internal Power Platform and Copilot Studio skills, and partners will continue to sell implementation and support services successfully. The risk is precisely that the existing model can remain profitable long enough to delay transformation.

Structural changes rarely make the previous business disappear immediately. They first alter margins, then customer expectations, then procurement behavior, then talent economics, until eventually the firms that invested early operate with structural advantages that become difficult to replicate quickly. The best moment for a partner to transform is therefore while the current model still generates the cash required to finance the next one.

Timing can be read from leading indicators rather than forecasts. Watch the estimate: when comparable engagements are being won at materially fewer days than two years ago, compression is already priced in. Watch procurement: when customers begin asking what AI has done to your rate card, the expectation has moved. Watch the pipeline: when the proportion of revenue that depends on the customer lacking internal capability grows rather than falls, dependency is being sold. Watch the bench: when the roles that are hardest to fill are process, architecture, and evaluation rather than development, the talent market has already repriced. Two or more of those moving together is the signal that the transition should already be funded.

Build process-consulting capability before the implementation business compresses, develop Executable Consulting IP before every competitor has equivalent AI delivery tooling, transform managed services before customers demand productivity discounts, create new career models before repetitive implementation work becomes difficult to staff, and develop Governance Intelligence before customers are overwhelmed by agentic risk. The partner is not being asked to abandon a successful business but to use today's successful business to build tomorrow's differentiation.

That requires leadership to accept a temporary tension, because investments in reuse reduce near-term billable effort, enablement makes customers more independent, better portfolio discipline stops projects that would have generated delivery revenue, and automated support reduces ticket volumes, all of which look economically irrational when viewed through the metrics of the old model and become rational the moment the goal is to build a more defensible firm.

That tension has to be funded explicitly, because it will never be funded implicitly. A practice that expects reusable capability to emerge from billable engagements will find that it does not: the people who could build it are the people with the least unallocated time. The practical mechanisms are few, and they amount

to a ring-fenced percentage of senior capacity held outside utilization targets, a capability team funded from practice margin rather than from projects, or a rule that a defined share of the productivity gain captured at the AI-enabled stage is reinvested rather than taken as margin. The mechanism matters less than the decision to name one.

The transition is therefore a strategic choice about timing. Partners can transform from a position of strength, or they can wait until the economics force them to change defensively, and the first path provides time to develop consulting capability, intellectual property, and new commercial models while the second requires transformation at exactly the moment when margins and confidence are already under pressure.

Conclusion - The Partner as a Force Multiplier for the Agentic Center of Excellence

Power Platform changed the economics of enterprise technology by making software creation more accessible, allowing people closer to the business to participate directly in solution development and pushing organizations to reconsider the boundary between central IT and distributed innovation. The Center of Excellence emerged as one of the organizational responses to that change, providing the structures required to govern, enable, and scale a technology ecosystem in which far more people could create solutions.

Agentic AI creates the next version of the same challenge, because technology is not merely becoming easier to build but is increasingly capable of participating in work: agents can interpret information, use knowledge, invoke tools, and execute portions of business processes, development systems can generate more of the solution from a description of intent, operational systems can diagnose and resolve increasing classes of incidents, and knowledge that previously existed only in documentation can increasingly be expressed as executable instructions and skills.

This does not eliminate the need for the Center of Excellence; it makes the CoE more important, while moving the value inside it. Strategy must address how human and digital work should be distributed, governance must respond to new forms of autonomy, innovation must become more selective because experimentation becomes easier, the Innovation Hub must invest more intelligence before implementation and make implementation itself increasingly industrial, enablement must build sustainable capability for a workforce collaborating with agents, and sustain must challenge operational work before increasing operational capacity.

The partner must follow the same movement. The traditional partner frequently solved scale problems by adding people, whereas the future partner should increasingly solve them by improving the system: when operational demand grows, the first question should not be how many analysts are required but why the demand exists and whether it can be eliminated, automated, or agentified; when innovation demand grows, the answer should not automatically be more developers but stronger idea nurture, better value assessment, better process design, and a more industrialized Innovation Hub; when governance complexity grows, the solution should not simply be more administrators but the judgment required to recognize new risks and convert that understanding into scalable controls; and when adoption is weak, the response should not simply be more training but an understanding of why behavior has not changed, and an enablement system capable of creating sustainable organizational capability.

Development remains important across all of it, although development itself changes character, because the valuable engineer increasingly creates leverage: one reusable tool rather than ten duplicated integrations, one evaluation framework rather than ten manual test exercises, one secure capability consumed by many agents rather than bespoke implementation in each solution. Engineering moves

toward the creation of reusable capability, and consulting moves toward the design of the environment in which that capability is used.

This is why the future Power Platform partner is not simply a more efficient implementation partner but a different kind of organization. It understands business processes deeply enough to challenge them and technology well enough to know what should be automated and what should not; it possesses enough architectural maturity to prevent democratized creation from becoming democratized fragmentation, enough Governance Intelligence to recognize risks that did not exist in yesterday's technology, enough experience-design capability to create trustworthy human-agent interaction, and enough institutional discipline to convert the expertise of its best consultants into reusable intellectual property.

Most importantly, it aligns its economics with customer improvement, because it does not need an inefficient customer in order to remain relevant: it creates a more capable customer and then remains valuable by continuing to bring insight the customer cannot easily reproduce alone, which is a fundamentally stronger consulting relationship. The transition can therefore be summarized through a single movement:

From Capacity Augmentation to Capability Augmentation.

The partner of the previous era adds people when the Center of Excellence needs to do more, while the partner of the agentic era helps the Center of Excellence redesign how the work is done so that additional people are not automatically the answer. Its value is found in judgment before execution, architecture before proliferation, process redesign before automation, governance before autonomy, sustainable adoption before training volume, and continuous optimization before operational scale.

That changes the talent the partner needs, the intellectual property the partner creates, how delivery is organized, how managed services are measured, and how the partner should price value, and ultimately it changes what it means to be a Power Platform partner. The future partner is not simply an external workforce surrounding the Center of Excellence; it is a force multiplier for the Center of Excellence, and its presence should make the CoE more intelligent in how it selects opportunities, more disciplined in how it designs solutions, more scalable in how it produces them, more adaptive in how it governs them, more effective in how it enables the organization, and more efficient in how it sustains the resulting estate.

That is the consulting role the agentic era creates, and it is why Power Platform partners must evolve.

Continue the Series

This white paper answered a single question: what kind of partner a Center of Excellence needs once execution becomes easy to generate and value moves toward judgment, architecture, and governance. The Building the Center of Excellence for Power Platform series addresses the wider framework that partner is there to strengthen.



Defining the Framework Structure for the Power Platform Center of Excellence

[View on Amazon ->](#)

<https://www.amazon.com/dp/B0GDDRC2C>



Copilot Studio and the Future of the Power Platform Center of Excellence

[View on Amazon ->](#)

<https://www.amazon.com/dp/B0H2SXQ3QQ>

Access to these and more material here:

<https://nfernandezba.github.io/Building-the-Center-of-Excellence-for-Power-Platform/>

